

Cooperative Multi-Agent Reinforcement Learning



Outline

- **Different Multiagent RL Paradigms**
- VDN
- QMIX



Different MARL Paradigms

- Independent Learners
- Centralized Training and Centralized Execution (CTCE)
- Centralized Training and Decentralized Execution (CTDE)
- Decentralized Training and Decentralized Execution (DTDE)

Setting

- Multiple agents
 - Each agent:
 - has **local observation** o_i
 - selects **action** a_i
 - Environment returns a **single shared reward**
- 👉 **Goal: maximize team reward**

Why is this hard?

- Credit assignment
- Non-stationarity
- Partial observability
- Exponential joint action space

CTCE

- **Centralized Training and Centralized Execution (CTCE)**

- Learn:

$$Q(s, a_1, \dots, a_n)$$

✗ Problem: action space explodes

Independent Learners

- Independent learners

- Each agent learns:

$$Q_i(o_i, a_i)$$

- ✗ Problems:

- non-stationarity
 - spurious rewards
 - poor coordination

Outline

- Different Multiagent RL Paradigms
- **VDN**
- QMIX



Key challenge

👉 We want:

- decentralized policies

👉 But we need:

- coordination from shared reward

Key idea of VDN

- Instead of learning:

$$Q_{total}(s, \mathfrak{a})$$

- Learn

$$Q_{total} \approx \sum_i Q_i(o_i, a_i)$$

- 👉 Decompose the value, not the reward

Important clarification

- Agents do NOT learn independently
- They do NOT receive individual rewards
- 👉 They learn jointly through a shared loss

Architecture (Concept)

- Each agent has a network:

$$Q_i(o_i, a_i; \theta_i)$$

- Outputs are combined:

$$Q_{total} = \sum_i Q_i$$

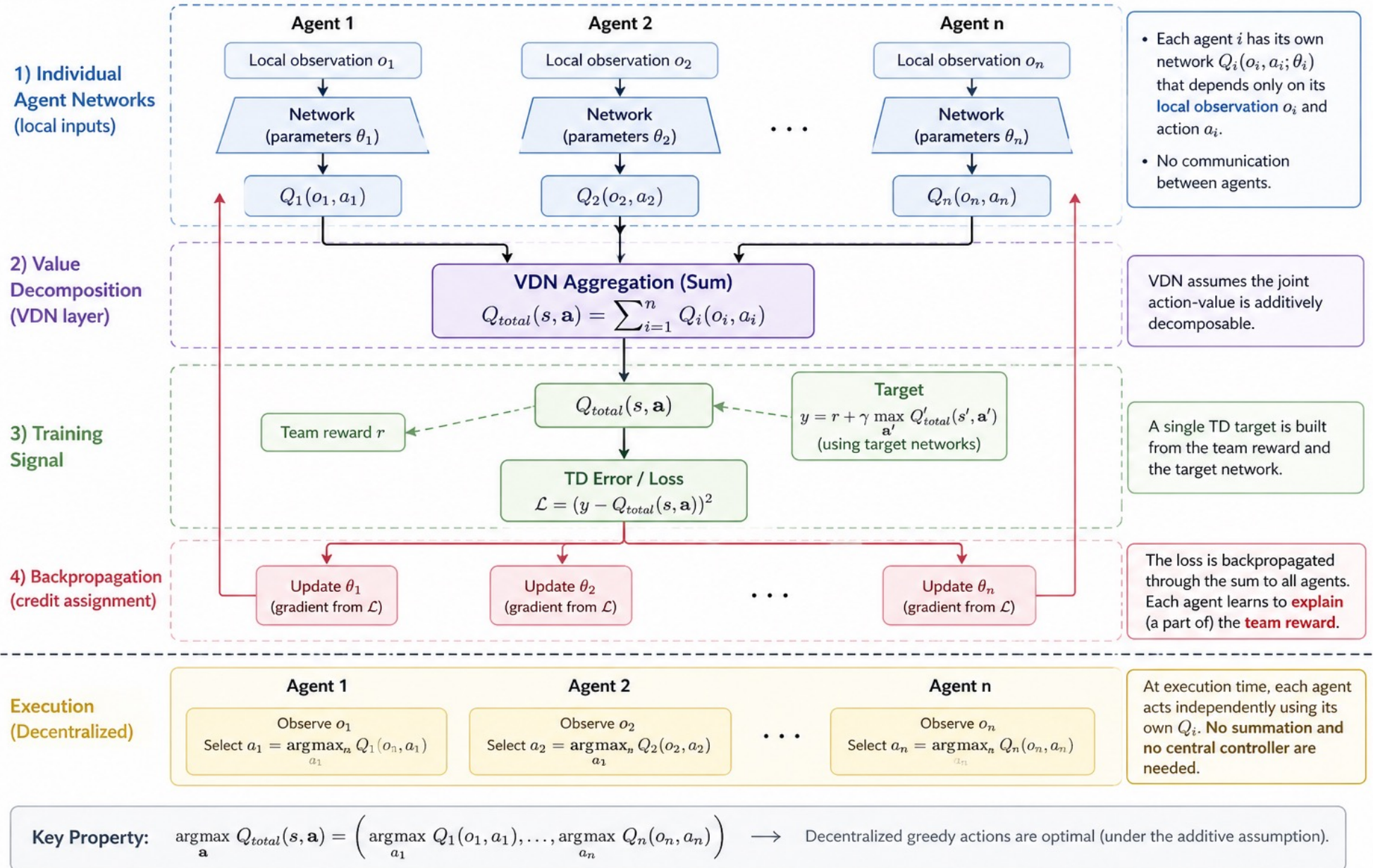
👉 Simple sum layer (VDN layer)

Training pipeline (big picture)

1. Each agent computes Q_i
2. Sum to get Q_{total}
3. Compute TD target
4. Compute loss
5. Backpropagate to all agents

👉 Centralized training

VDN: Training Pipeline (Centralized Training, Decentralized Execution)



Target network (very important)

Each agent has:

- Online network:

$$Q_i(o_i, a_i; \theta_i)$$

- Target network:

$$Q_i(o_i, a_i; \theta_i^-)$$

👉 θ_i^- is a **delayed copy** of θ_i

Why target networks?

- Stabilize learning
- Avoid moving target problem
- 👉 Same idea as DQN

TD Target (with target network)

- We compute:

$$y = r + \gamma \max_{a'} Q_{total}^-(s', a')$$

- Where:

$$Q_{total}^- = \sum_i Q_i(o_i', a_i'; \theta_i^-)$$

👉 Uses target networks

Loss function

- The loss function for VDN:

$$\mathcal{L} = (y - Q_{total})^2$$

👉 Only one loss for the whole system

Backpropagation

- Loss is computed on Q_{total}
- Gradients flow through the sum
- 👉 Each agent receives:
 - same team-level error signal

Backpropagation

- Gradient w.r.t. total value (Q_{total})

$$\frac{\partial \mathcal{L}}{\partial Q_{\text{tot}}} = -2(y - Q_{\text{tot}})$$

👉 This is the global TD error signal

Gradient to each agent (credit assignment)

- Apply chain rule:

$$\frac{\partial \mathcal{L}}{\partial \theta_i} = \underbrace{\frac{\partial \mathcal{L}}{\partial Q_{\text{tot}}}}_{\text{TD error}} \cdot \underbrace{\frac{\partial Q_{\text{tot}}}{\partial Q_i}}_{\text{credit assignment}} \cdot \underbrace{\frac{\partial Q_i}{\partial \theta_i}}_{\text{agent learning}}$$

👉 However:

$$Q_{\text{tot}} = \sum_i Q_i \Rightarrow \frac{\partial Q_{\text{tot}}}{\partial Q_i} = 1$$

Gradient to each agent (credit assignment)

- Final gradient expression:

$$\frac{\partial \mathcal{L}}{\partial \theta_i} = \frac{\partial \mathcal{L}}{\partial Q_{\text{tot}}} \cdot \frac{\partial Q_i}{\partial \theta_i}$$

👉 Every agent receives exactly the same TD error

👉 There is no weighting:

- no state dependence
- no interaction modeling

Key insight

- 👉 Agents are **not told** their contribution
- 👉 They **learn to explain the team reward**

Execution

- During execution:

$$a_i = \arg \max_{a_i} Q_i(o_i, a_i)$$

👉 No:

- summation
- central controller

Magic property

- Due to the assumption:

$$Q_{total} = \sum_i Q_i$$

We get:

$$\arg \max_{\mathbf{a}} Q_{total} = (\arg \max_{a_1} Q_1, \dots, \arg \max_{a_n} Q_n)$$

👉 Decentralized greedy = optimal joint action

What VDN fixes

👉 When **compared to independent learners and CTCE:**

- reduces credit assignment ambiguity
- improves stability vs independent learners
- avoids exponential joint action space

Limitations

$$Q_{total} = \sum_i Q_i$$

👉 Assumes additivity

✗ Cannot represent:

- complex interactions
- nonlinear coordination

Main Track Extended Abstract

AAMAS 2018, July 10-15, 2018, Stockholm, Sweden

Value-Decomposition Networks For Cooperative Multi-Agent Learning Based On Team Reward

Extended Abstract

Peter Sunehag, Guy Lever, Audrunas Gruslys, Wojciech Marian Czarnecki, Vinicius Zambaldi,
Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z. Leibo, Karl Tuyls, Thore Graepel
DeepMind, London, United Kingdom
sunehag@google.com

Value-Decomposition Networks For Cooperative Multi-Agent Learning

Peter Sunehag
DeepMind
sunehag@google.com

Guy Lever
DeepMind
guylever@google.com

Audrunas Gruslys
DeepMind
audrunas@google.com

Wojciech Marian Czarnecki
DeepMind
lejlot@google.com

Vinicius Zambaldi
DeepMind
vzambaldi@google.com

Max Jaderberg
DeepMind
jaderberg@google.com

Marc Lanctot
DeepMind
lanctot@google.com

Nicolas Sonnerat
DeepMind
sonnerat@google.com

Joel Z. Leibo
DeepMind
jzl@google.com

Karl Tuyls
DeepMind & University of Liverpool
karltuyls@google.com

Thore Graepel
DeepMind
thore@google.com

Outline

- Different Multiagent RL Paradigms
- VDN
- **QMIX**



Key idea of QMIX

👉 Replace simple sum with **learned mixing function**

$$Q_{total} = f(Q_1, \dots, Q_n; s)$$

- f is a neural network
- conditioned on global state s

QMIX decomposition

$$Q_{total} = f(Q_1, \dots, Q_n; s)$$

Key differences from VDN:

- not additive
- depends on state
- learned function

Key constraint

- QMIX imposes:

$$\frac{\partial Q_{total}}{\partial Q_i} \geq 0$$

- 👉 The total value is monotonically increasing in each agent's value
 - i.e., “If an agent's Q-value increases, the total value (Q_{total}) cannot go down.”

Intuition

👉 If an agent improves its action (higher Q_i)

👉 The team value should not decrease

🚫 Not allowed:

- “Agent improves $\rightarrow$ team gets worse”

Why Monotonicity?

- We want decentralized execution:

$$a_i = \arg \max_{a_i} Q_i(o_i, a_i)$$

👉 Each agent acts independently

Why monotonicity matters

- Monotonicity ensures:

$$\arg \max_{\mathbf{a}} Q_{total} = (\arg \max_{a_1} Q_1, \dots, \arg \max_{a_n} Q_n)$$

👉 Local greedy actions = global optimal action

QMIX architecture

- Each agent:
 - computes $Q_i(o_i, a_i)$
- Mixing network:
 - combines them into Q_{total}
- Hypernetwork:
 - generates weights using state s

Mixing Network

- What it does:
 - input: agent values Q_i
 - output: team value

$$Q_{total} = f(Q_1, \dots, Q_n)$$

👉 Combines individual values into global value

- Usually a Multi-Layer Perceptron (MLP)
 - weights are not just ordinary free parameters

Hypernetwork

- Usually a Multi-Layer Perceptron (MLP)
 - Weights are constrained to be non-negative (enforce monotonicity)
 - What it does:
 - input: global state s
 - output: weights/biases for the mixing network
- 👉 Purpose: make the mixing function state-dependent

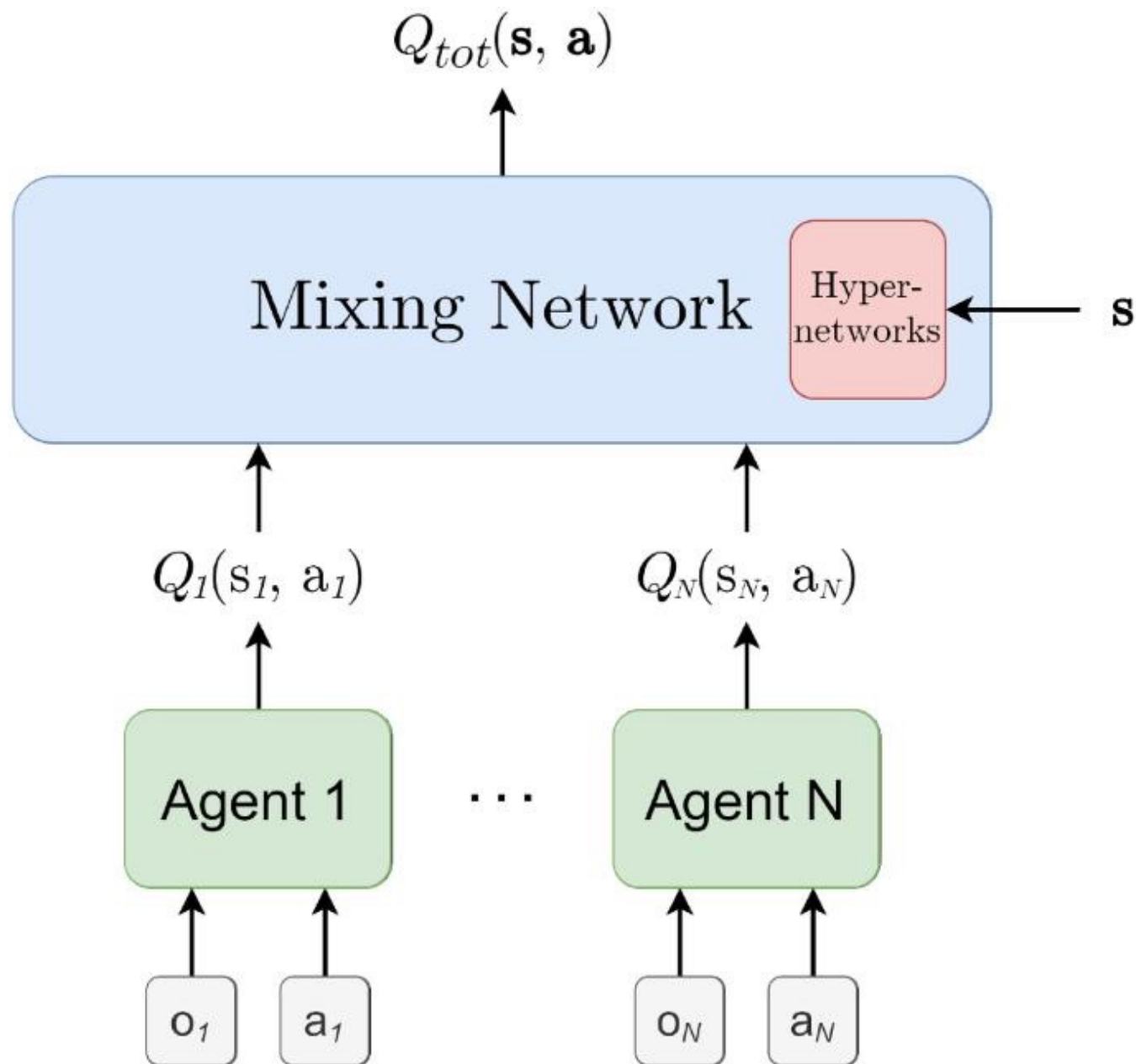
Example

$$Q_{total} = w_1 Q_1 + w_2 Q_2$$

- State s_1 :
 - $w_1 = 0.6$ and $w_2 = 0.1$
- State s_2 :
 - $w_1 = 0.2$ and $w_2 = 0.7$

Why this matters

- Enables:
 - state-dependent coordination
 - flexible credit assignment
- 👉 Different agents matter in different situations



Training Objective

- We compute:

$$y = r + \gamma \max_{\mathbf{a}'} Q_{total}^-(s', \mathbf{a}')$$

- Where:

$$\mathcal{L} = (y - Q_{total})^2$$

👉 Same as DQN / VDN

Target Networks

- Each agent:
 - online + target network
 - Mixing network:
 - also has target version
- 👉 Same idea as DQN

Backpropagation

- Gradient w.r.t. total value (Q_{total})

$$\frac{\partial \mathcal{L}}{\partial Q_{\text{tot}}} = -2(y - Q_{\text{tot}})$$

👉 This is the global TD error signal

Gradient to each agent (credit assignment)

- Apply chain rule:

$$\frac{\partial \mathcal{L}}{\partial \theta_i} = \underbrace{\frac{\partial \mathcal{L}}{\partial Q_{\text{tot}}}}_{\text{TD error}} \cdot \underbrace{\frac{\partial Q_{\text{tot}}}{\partial Q_i}}_{\text{credit assignment}} \cdot \underbrace{\frac{\partial Q_i}{\partial \theta_i}}_{\text{agent learning}}$$

👉 Interpretation:

$\frac{\partial Q_{\text{tot}}}{\partial Q_i}$ = how much agent i influences the team value

Gradient to hypernetwork

- Apply chain rule:

$$\frac{\partial \mathcal{L}}{\partial \phi} = \underbrace{\frac{\partial \mathcal{L}}{\partial Q_{\text{tot}}}}_{\text{TD error}} \cdot \underbrace{\frac{\partial Q_{\text{tot}}}{\partial W}}_{\text{mixing sensitivity}} \cdot \underbrace{\frac{\partial W}{\partial \phi}}_{\text{state-dependent learning}}$$

👉 Interpretation:

$\frac{\partial Q_{\text{tot}}}{\partial W}$ = how sensitive the mixing function is to its weights

$\frac{\partial W}{\partial \phi}$ = how the hypernetwork changes those weights

Why better than VDN

- captures interactions
- adapts to state
- still scalable

Limitation

- Monotonic constraint:

$$\frac{\partial Q_{total}}{\partial Q_i} \geq 0$$

👉 Cannot model:

- **negative interactions:** increasing one agent's local value Q_i can make the global team value Q_{total} lower
- **complex dependencies**

Simple Example of Negative Interaction

- Two robots try to pass through a narrow corridor.
 - If only Robot 1 enters, that may be good.
 - If only Robot 2 enters, that may also be good.
- But if both enter at the same time, they block each other.
 - So Robot 1's action may look locally good, and Robot 2's action may also look locally good, but together they produce a bad outcome.

QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning

Tabish Rashid^{*1} Mikayel Samvelyan^{*2} Christian Schroeder de Witt¹
Gregory Farquhar¹ Jakob Foerster¹ Shimon Whiteson¹

Abstract

In many real-world settings, a team of agents must coordinate their behaviour while acting in a decentralised way. At the same time, it is often possible to train the agents in a centralised fashion in a simulated or laboratory setting, where global state information is available and communication constraints are lifted. Learning joint action-values conditioned on extra state information is an attractive way to exploit centralised learning, but the best strategy for then extracting decentralised policies is unclear. Our solution is QMIX, a novel value-based method that can train decentralised policies in a centralised end-to-end fashion. QMIX employs a network that estimates joint action-values as a complex non-linear combination of per-agent values that condition only on local observations. We structurally enforce that the joint-action value is monotonic in the per-agent values, which allows tractable maximisation of the joint action-value in off-policy learning, and guarantees consistency between the centralised and decentralised policies. We evaluate QMIX on a challenging set of StarCraft II micromanagement tasks, and show that QMIX significantly outperforms existing value-based multi-agent reinforcement learning methods.

1. Introduction

Reinforcement learning (RL) holds considerable promise to help address a variety of cooperative multi-agent problems, such as coordination of robot swarms (Hüttenrauch et al., 2017) and autonomous cars (Cao et al., 2012).

^{*}Equal contribution ¹University of Oxford, Oxford, United Kingdom ²Russian-Armenian University, Yerevan, Armenia. Correspondence to: Tabish Rashid <tabish.rashid@cs.ox.ac.uk>, Mikayel Samvelyan <mikayel@samvelyan.com>.

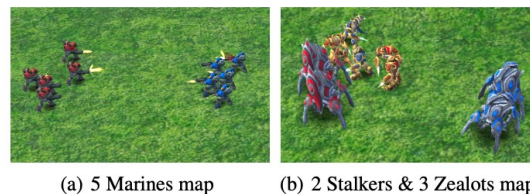


Figure 1. Decentralised unit micromanagement in StarCraft II, where each learning agent controls an individual unit. The goal is to coordinate behaviour across agents to defeat all enemy units.

In many such settings, partial observability and/or communication constraints necessitate the learning of *decentralised policies*, which condition only on the local action-observation history of each agent. Decentralised policies also naturally attenuate the problem that joint action spaces grow exponentially with the number of agents, often rendering the application of traditional single-agent RL methods impractical.

Fortunately, decentralised policies can often be learned in a centralised fashion in a simulated or laboratory setting. This often grants access to additional state information, otherwise hidden from agents, and removes inter-agent communication constraints. The paradigm of *centralised training with decentralised execution* (Oliehoek et al., 2008; Kraemer & Banerjee, 2016) has recently attracted attention in the RL community (Jorge et al., 2016; Foerster et al., 2018). However, many challenges surrounding how to best exploit centralised training remain open.

One of these challenges is how to represent and use the action-value function that most RL methods learn. On the one hand, properly capturing the effects of the agents' actions requires a centralised action-value function Q_{tot} that conditions on the global state and the joint action. On the other hand, such a function is difficult to learn when there are many agents and, even if it can be learned, offers no obvious way to extract decentralised policies that allow each agent to select only an individual action based on an individual observation.

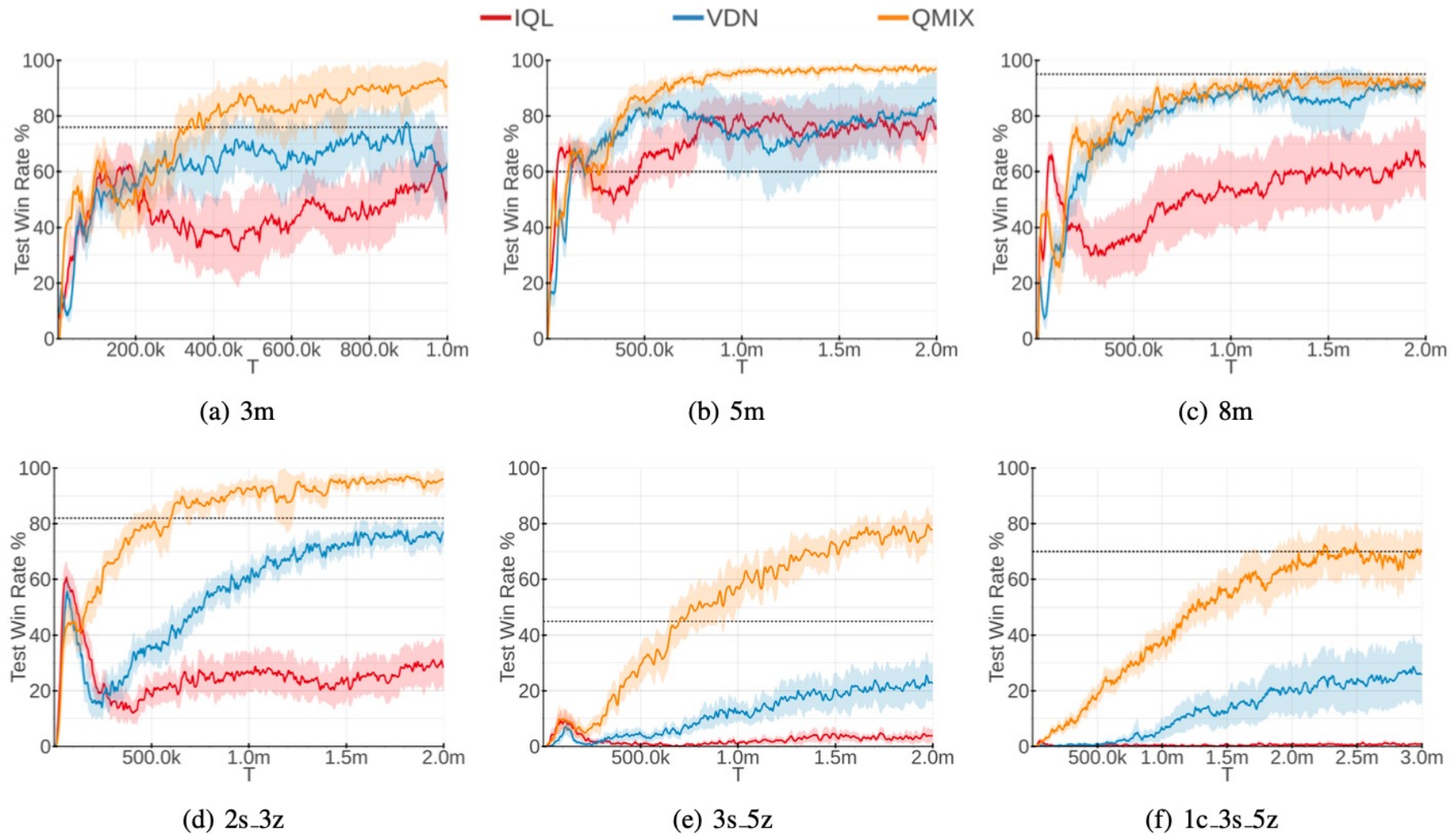


Figure 3. Win rates for IQL, VDN, and QMIX on six different combat maps. The performance of the heuristic-based algorithm is shown as a dashed line.

Thank You



sardinha@inf.puc-rio.br